JustRUA

Controlled Study of GPT-6 RUAs

JustRUA: A Controlled Study of GPT-6 Robot-Use Agents

Zefan Wang, Yufan Dang, …, Xin Cong, Yuan Yao
Tsinghua Univ. · HUST · Peking Univ. · ModelBest · Tencent
arXiv 2611.16009 · First submitted 2026-11-11

RoboDojo eval-set total score

29.7

↓

50.3

No learned policy · no task tuning

Paper Review

One-Sentence Summary

Lean harness + cheap geometry: score rises from 29.7 to 50.3

Eval-set total score (111 episodes, 42 tasks)

29.7 → 50.3

Success rate 22.5% → 43.3%

No learned policy (VLA)
No self-improvement loop
No task-specific tuning

Kept in JustRUA (5)

- Full episode history (all images)
- Each action runs ⩽ 20 control steps
- Compact text gripper readings
- Depth Pro depth maps predicted from RGB
- Code solves 3D coordinates from pixels

Tried but not kept

- Action interface: relative goals / semantic skills / self-chosen steps / action sequences
- Experience reuse: visual demos / text trajectories / self-built skill library
- Precomputed affordances (4 presentations)

The supposed "weak precision of GPT-6 Astra" lies largely in the harness, not the model: a plain interface plus targeted, compact geometry is enough.

Problem: RUA scores climb fast, yet gains are hard to attribute

What Is an RUA?

Robot-Use Agent

- GPT-6 Astra views three camera images and directly outputs end-effector target poses
- The non-learned planner CuRobo generates joint trajectories
- No VLA in between, akin to a computer-use agent that "looks at the screen and clicks"

Tension

"Weak model" or "weak harness"?

- Under the official L3 harness, GPT-6 Astra is weak on precision tasks
- A few reasonable changes bring its Precision score close to the then-best VLA (DM0.5)

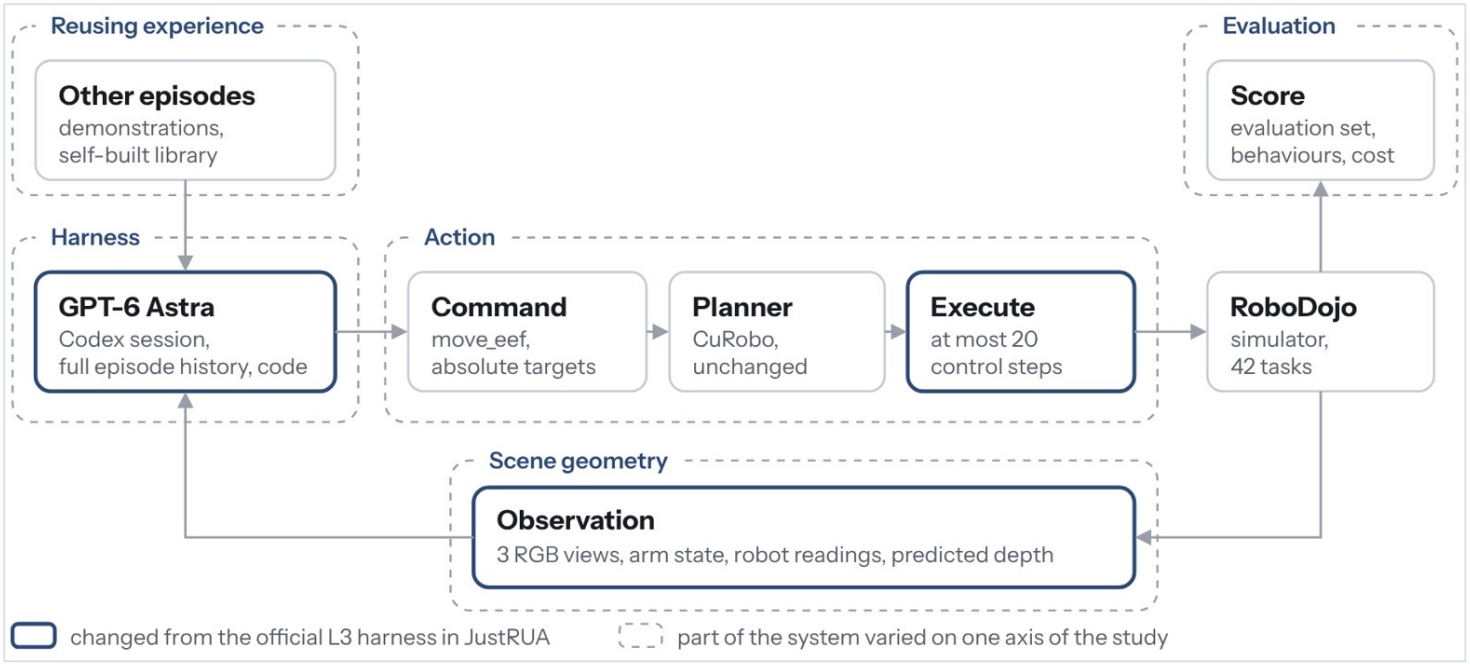
Hard to Compare

Systems change too much at once

- Harnesses, test subsets and budgets all differ
- Depth, affordances, demos, self-improvement, etc. are often added at once
- Scores rise, but which component helps is unknown

This work: instead of reproducing each system, decompose the designs and compare them one by one on the same testbed, eval set and cost accounting

One loop, four axes: one change at a time, kept only if worthwhile



Solid blue boxes mark JustRUA's changes to the official L3 harness; each dashed region is one experimental axis; the CuRobo planner and simulator stay fixed.

1 Harness
History retained, reasoning effort

2 Action
Command format, duration per action

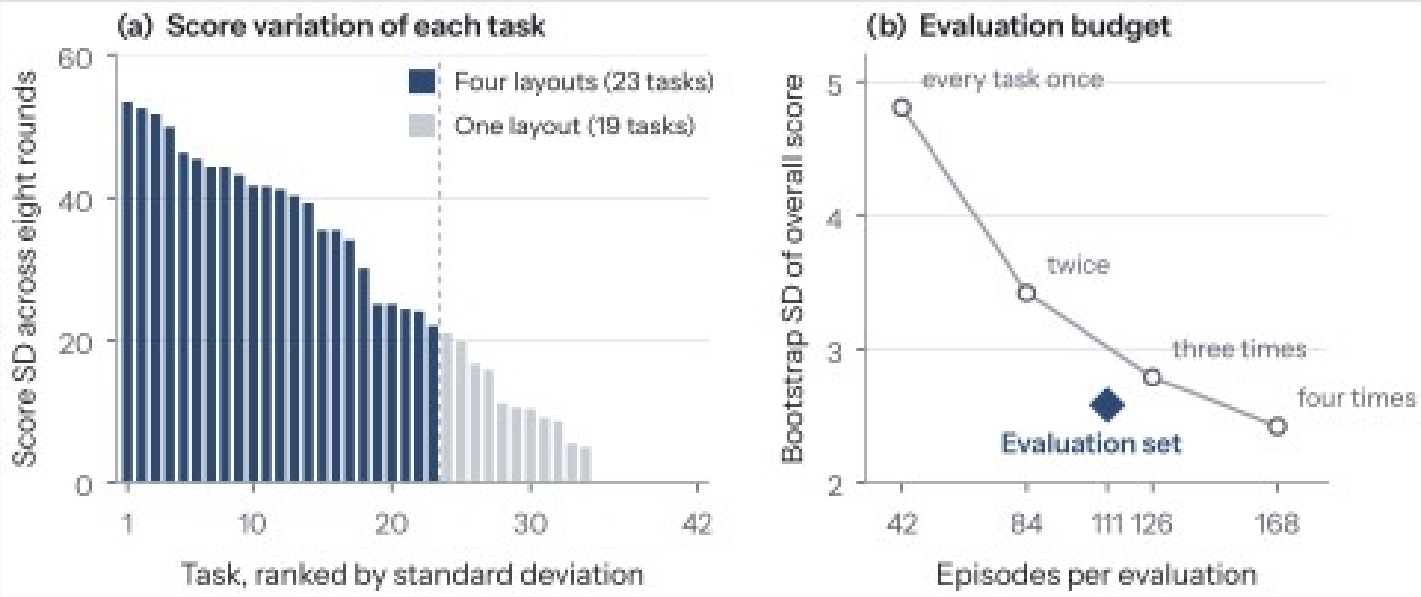
3 Reusing experience
Demos → text traj. → self-written skills

4 Scene geometry
Readings→depth→afford.→code

Retention criteria (all three required)

- Helps on the eval set
- Reasonable cost
- Uses no info from evaluated tasks

111-episode eval set is more stable than 3 runs/task (126 episodes)



(a) Per-task score std. over 8 rounds for 42 tasks; the 23 most variable run 4 layouts; (b) bootstrap std. of the total score at different budgets.

23×4 + 19×1 = 111

More runs for high-variance tasks; fixed layouts shared by all configs

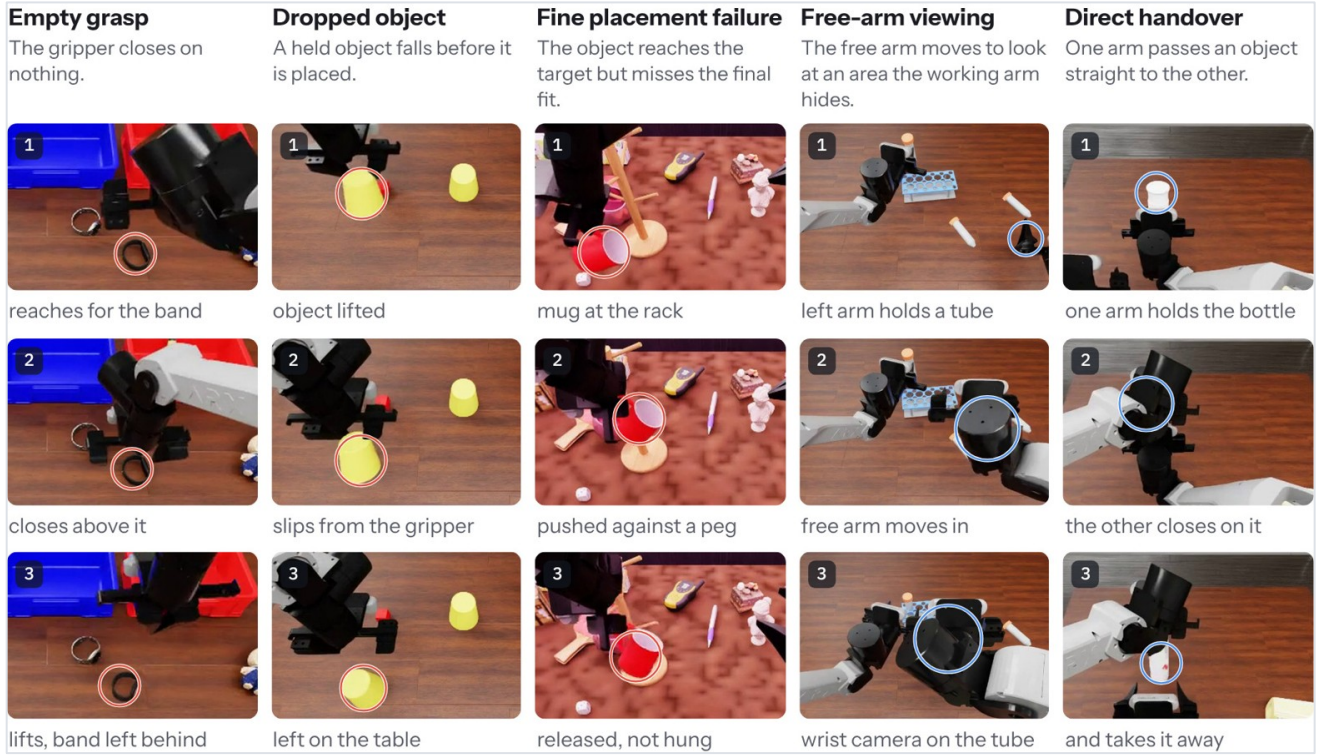
Bootstrap std. of total score (medium reasoning)

Protocol	Episodes	Std.
1 per task	42	4.8
3 per task	126	≈2.8
Our eval set	**111**	**2.6**
4 per task	168	2.4

29.7 vs 29.0

Official harness: eval-set score vs. full 2,100-episode evaluation

Beyond scores: audit behavior per episode; cost at cached prices



Behavior Audit

An independent GPT-6.1 Sol audits 3-camera videos and full action logs of 42 first-round episodes, then re-checks; only frame-evidenced events count (lower bounds)

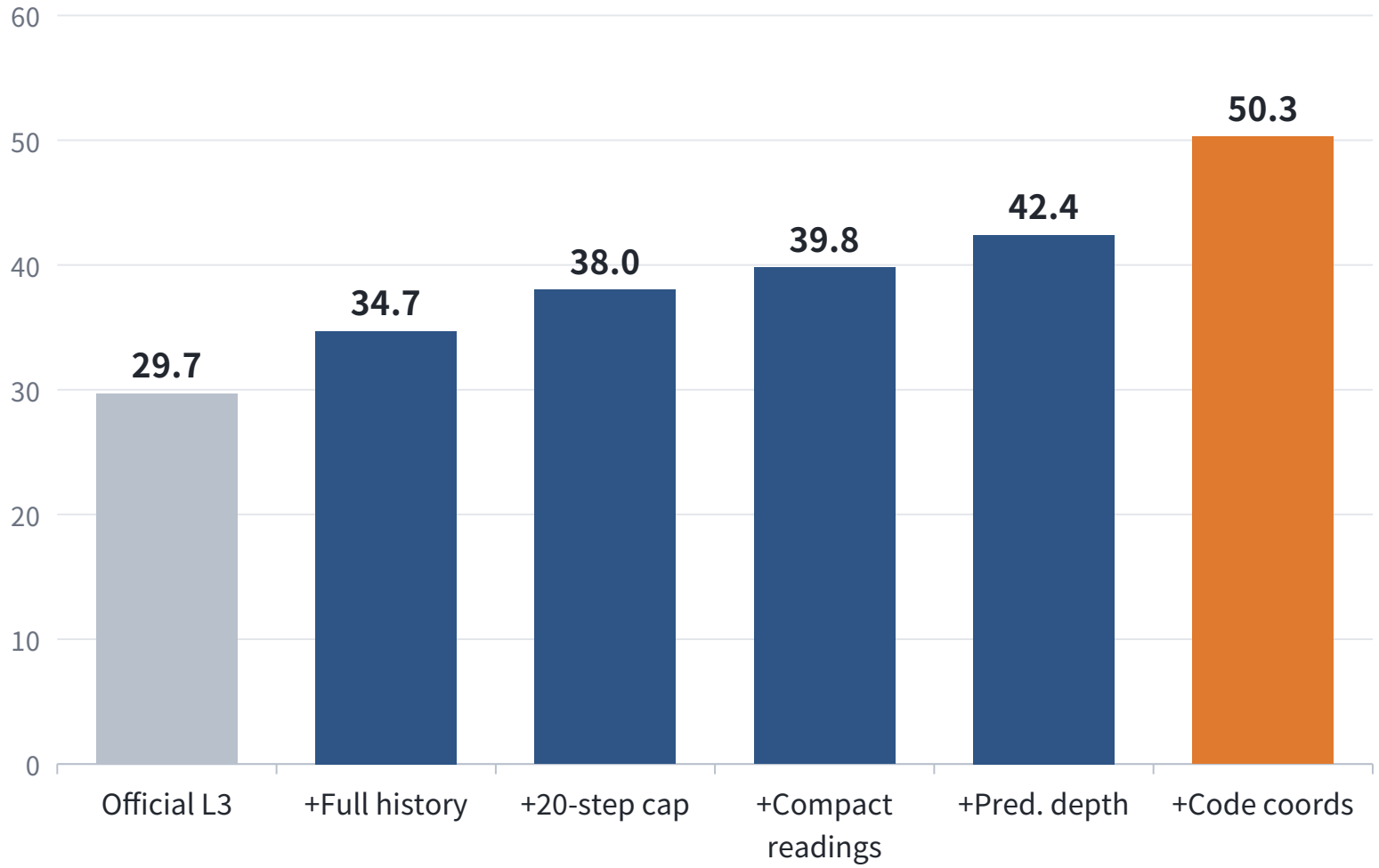
Behaviors Counted

Empty grasps, drops, fine-placement failures (failures); view adjustments, handovers (neutral); plus unintended collisions

Cost Accounting

Each decision resends full history (millions of input tokens per episode); first-round cost per episode is a lower bound with all cacheable input cached

Kept path: biggest gains from early harness and final code coords

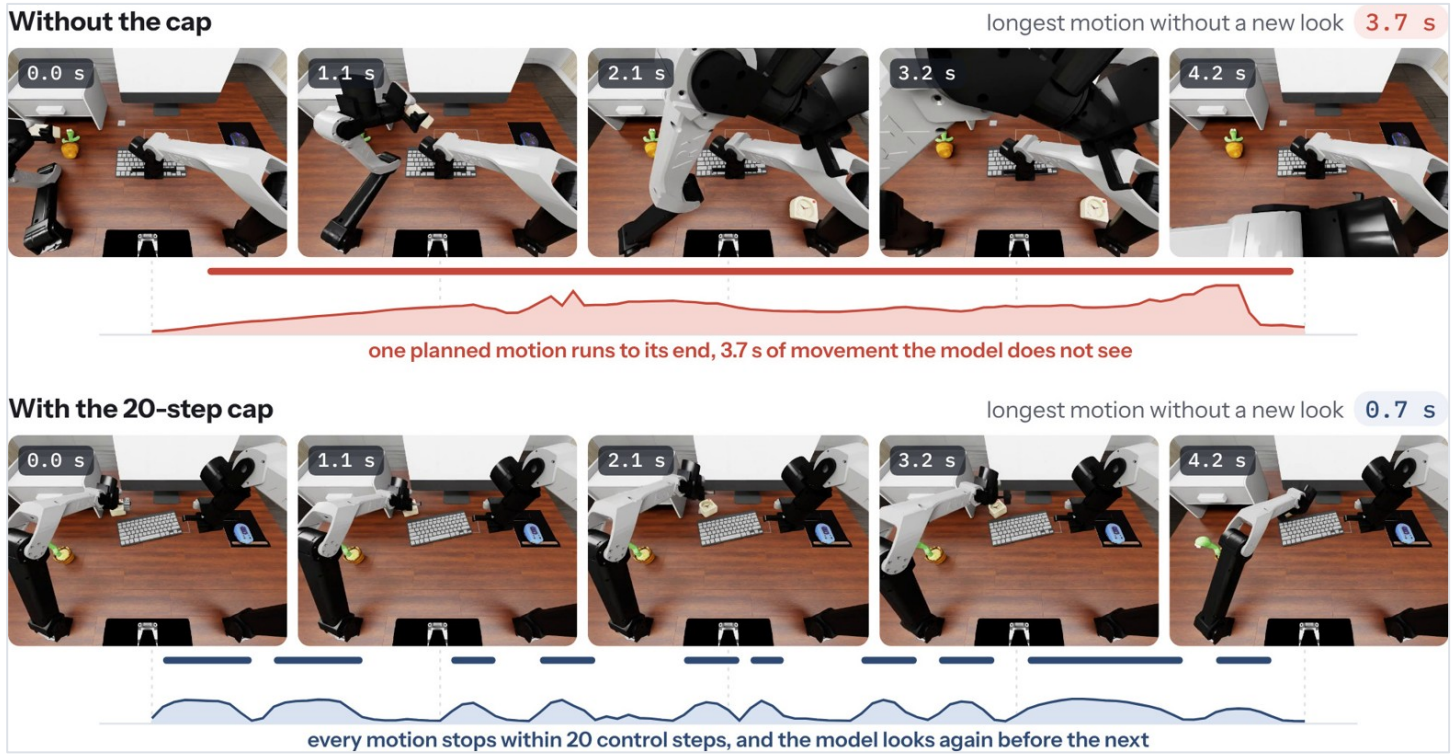


Gain and cost per step

+5.0	Full history	$3.76
+3.3	20-step cap	$4.94
+1.8	Compact readings	$5.51
+2.6	Predicted depth	$7.74
+7.9	Code coords	$12.53

Costs are first-round per-episode lower bounds (all-cached prices). The final step also removed the 100-decision cap; the two contributions cannot be separated.

Harness: keep full history + stop each action early at 20 steps



organize_table, same 4.2 s: uncapped, one unstable plan sweeps the arm across the table, blocking the camera; the model never sees it.

3.7 s → 0.7 s

Longest continuous motion the model "cannot see"

Precision 9.4 → 25.0

The 20-step cap's gain concentrates in Precision

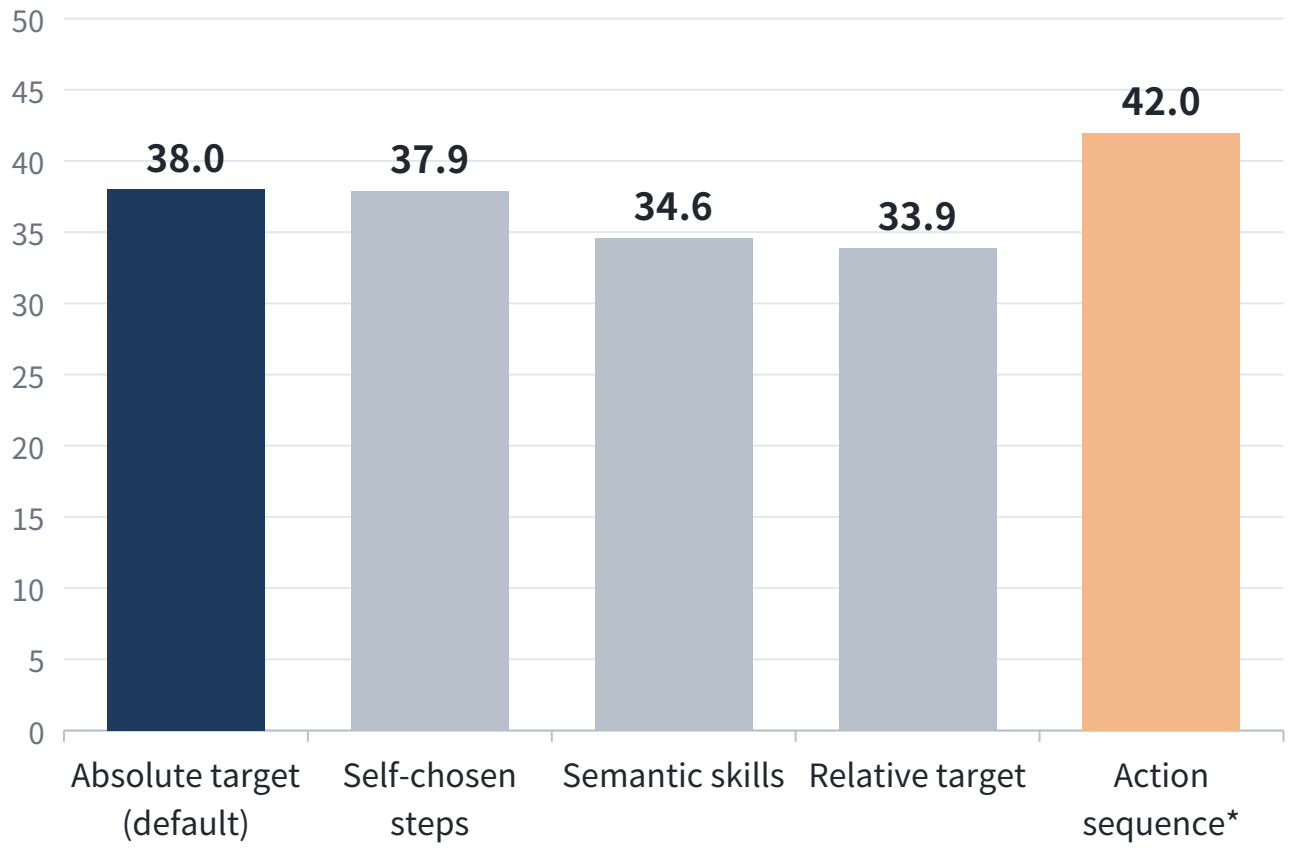
29.7 → 34.7 → 38.0

Full history, then 20-step cap (eval-set total)

Truncate without resampling; the rest of the trajectory is dropped; feedback still reports error vs. the original target and steps actually executed.

Merely moving to Codex without keeping history does not help by itself.

“Smarter” action interfaces did not beat plain absolute targets



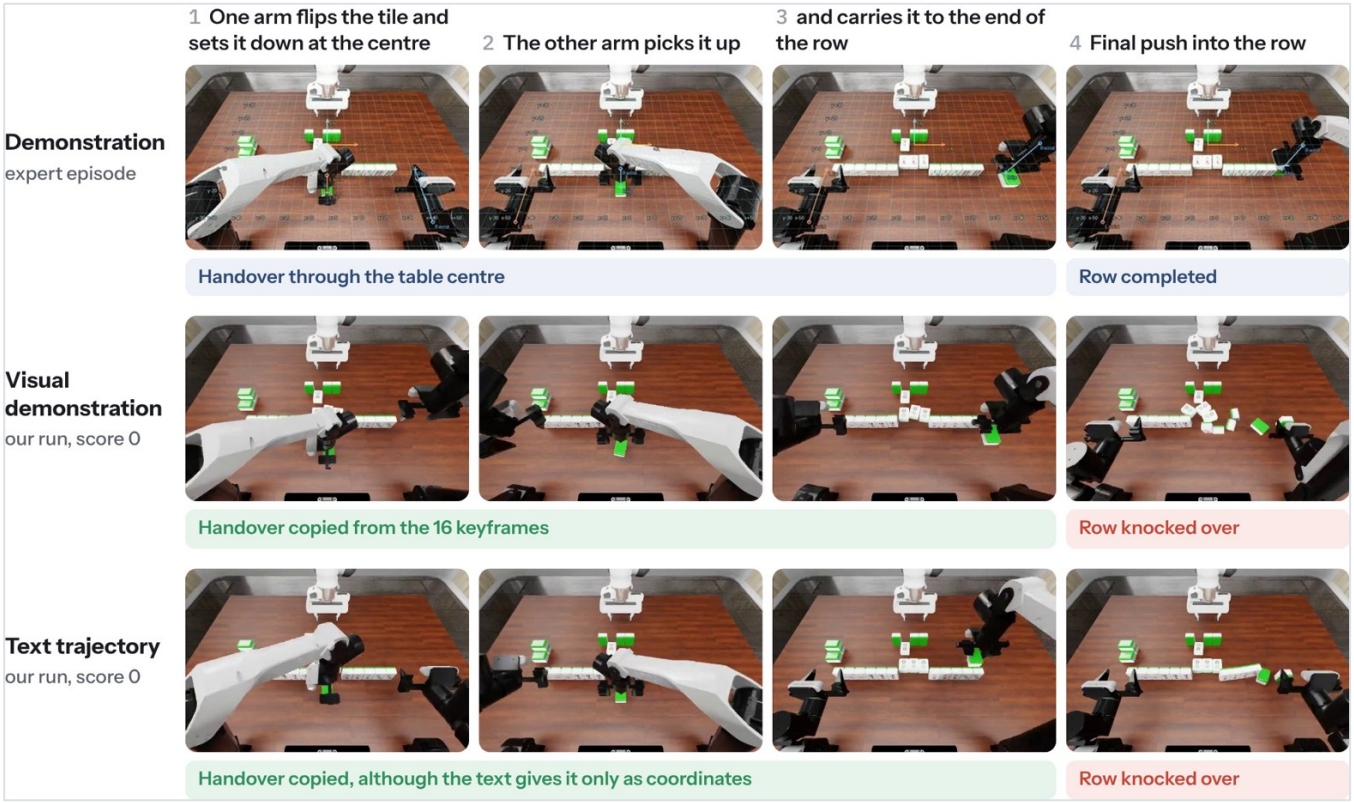
* Action-sequence gains come mainly from 6 episodes that had exhausted the 100-decision budget; had they kept their original scores, the total would be only 39.4, so JustRUA simply removes the budget.

Behavior counts, 42 first-round episodes

Behavior	Absolute	Relative	Semantic	Sequence
Empty grasp	47	54	47	38
Drop	26	30	35	27
View adjust.	59	54	211	45
Arm hits object	7	20	12	1

- **Relative targets:** ~3× more object collisions, possibly since all inputs are world coordinates, needing an extra subtraction
- **Semantic skills:** 3×+ more view adjustments and more drops; changes “what” is done, not “how well”
- **Reasoning effort:** medium is best; low scores only 5.3 on Precision

Experience reuse changed behavior but not scores: no free lunch



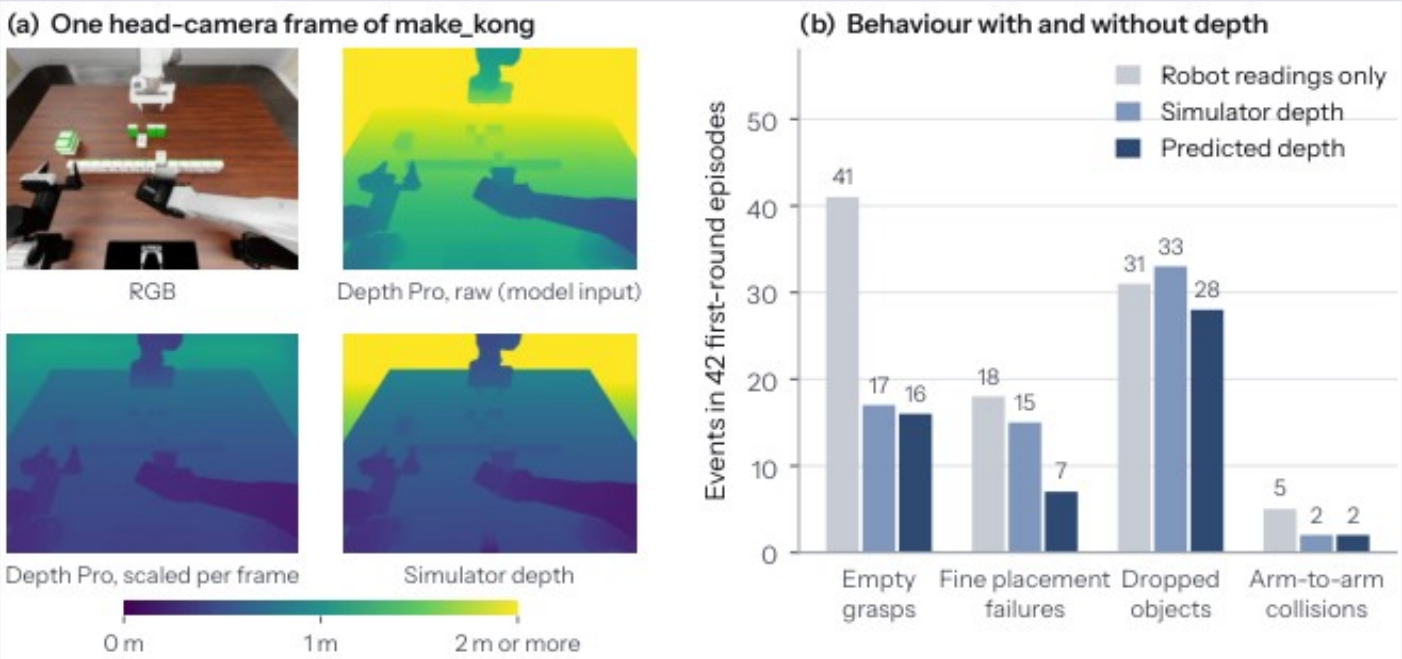
make_kong: both demo types teach handovers via the table center, but both knock over the whole row in the final push-in, scoring 0.

Config	Eval set	Cost $
Default testbed	38.0	4.94
Visual demos	39.9	5.11
Text trajectories	35.2	5.84
Self-built skills	37.0	5.43

- Demos: more handovers; drops 26 → 20 / 17
- Skill library: empty grasps 47 → 33, inter-arm collisions 6 → 0
- Fix one stage, and episodes often fail elsewhere

Experience comes from evaluated tasks, so scores are not comparable to standard eval; JustRUA keeps none

Predicted depth is off by tens of cm, yet as useful as sim depth



(a) Head-camera RGB, raw Depth Pro prediction (actual model input), per-frame scaled result and sim depth; (b) behavior counts in three settings.

Depth Pro accuracy (head camera)

Setting	MAE	r
Raw prediction (model input)	**73.7 cm**	**0.993**
True focal length	40.1 cm	0.993
Per-frame scaling	7.3 cm	0.993

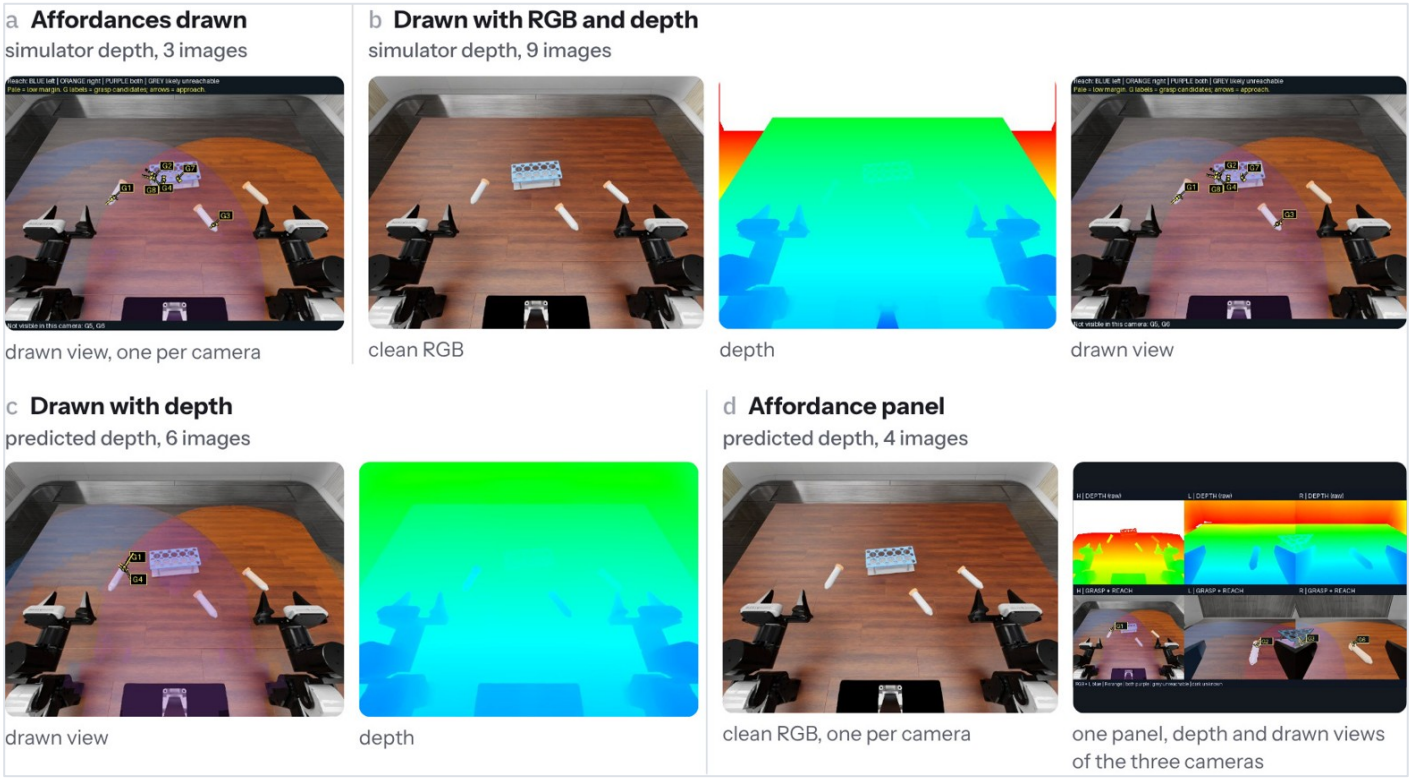
41 → 16
First-round empty grasps

18 → 7
Fine-placement failures

Total score: compact readings 39.8 → predicted depth 42.4 (sim depth: 46.0, reference only; not accepted by the leaderboard)

Insight: depth only needs the right "relative order"; this gain needs no privileged information

No gain from precomputed affordances: rarely useful, paid every step

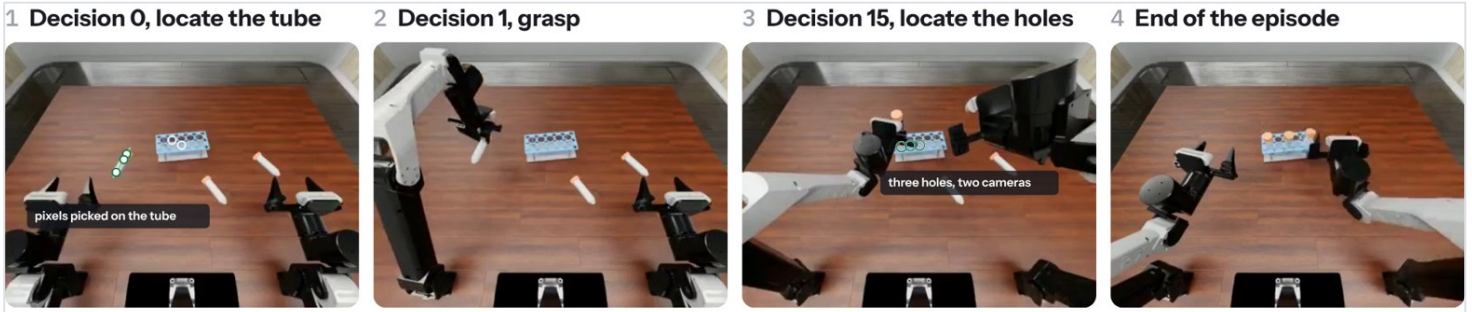


Config	Eval set	Empty grasp
Sim depth	46.0	17
+ Annot. (sim depth)	36.7	17
Predicted depth	**42.4**	**16**
+ Annot. (pred. depth)	30.9	39
+ Summary panel (pred.)	42.1	34

- Vs. same-source depth, none of the four grasps more reliably or scores higher
- Reachability helps only in handovers, which remain rare
- Grasp candidates help only when initiating a grasp, yet cost is paid at every observation

Four presentations: annotations on views / RGB & depth re-added / predicted depth + annotations / summary panel (GraspGen grasp candidates + IK-reachable regions)

Model writes code to solve 3D coords from pixels: score hits 50.3



The model casts rays from pixels of the left tube through the head camera onto the table plane.

The left gripper goes to the solved point, **1.2 mm from the tube axis**, closes and lifts the tube.

Three holes of the rack and the tube tip are triangulated from the head and right wrist cameras.

All three tubes stand in the rack, **score 100**. A later decision reuses a solved hole, 0.4 mm off.

CODE THE MODEL RUNS AT DECISION 0

```
import numpy as np
for u,v in [(220,202),(201,235),(215,212),(302,177),(319,186)]:
 d=np.array([(u-320)/288.1325,-.86603*(v-240)/288.1325+.5,
             -.5*(v-240)/288.1325-.86603])
 p=np.array([0,-.41,1.308]); q=p+d*((.782-p[2])/d[2]); print(u,v,q)
```

OUTPUT

```
220 202 [-0.22816849 -0.00619792  0.782]
201 235 [-0.2533848  -0.09402191  0.782]
215 212 [-0.23449104 -0.03411066  0.782]
302 177 [-0.04342494  0.0691849   0.782]
319 186 [-0.00236371  0.04107148  0.782]

→ decision 1: left_x -0.232, left_y -0.024
```

93.2%
Code runs used for geometry (123/132)

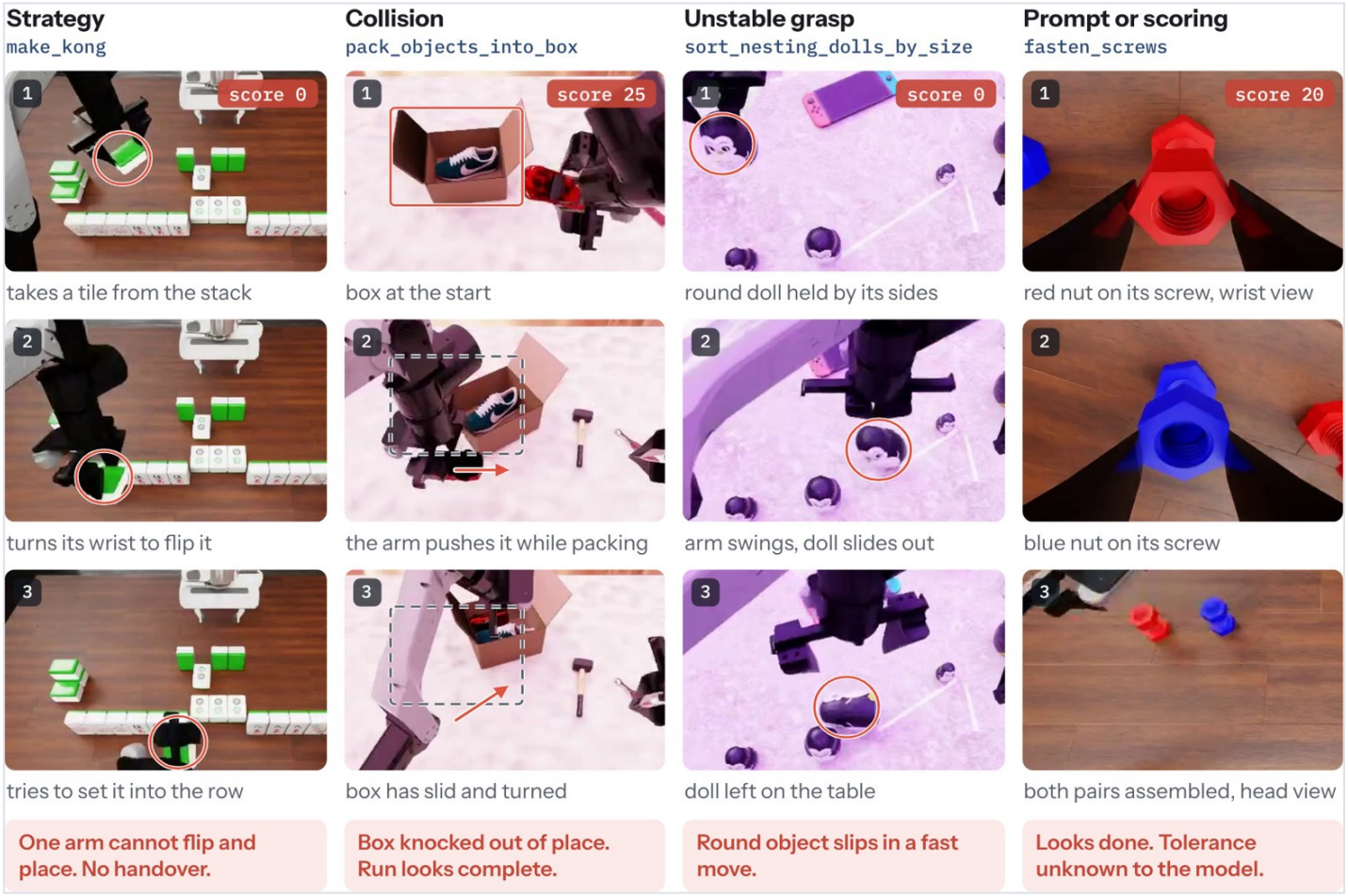
40 → 100
insert_tubes, same layout: decisions 65 → 57

Long-hor. 31.6 → 52.2
Largest gain; Memory 55.6 → 70.8

$12.53 / episode
Costliest step; also removes 100-decision cap

insert_tubes: pixel rays intersected with the table; holes triangulated from two cameras (distance = code output vs. later command, not vs. ground truth)

Per-task long tail remains: 21/42 first-round tasks score below 50



8 **Strategy (tasks)**

Plan unsuited to the arms: make_kong lacks handover, tips forward

6 **Collision (tasks)**

Misjudged arm or held-object size: light boxes pushed away

6 **Unstable grasp (tasks)**

Round/irregular objects slip: nesting dolls, coins

5 **Prompt/scoring (tasks)**

Looks done, but the model doesn't know the tolerance: fasten_screws

Prompts rewritten to scoring criteria: first round 54.0 → 55.6 (exploits the grader; excluded from JustRUA)

Further gains need per-task trial and error, close to tuning on test tasks

Two themes: plain interfaces suffice; info must be apt and compact

GPT-6 Astra: plain I/O works

World-frame absolute targets, raw sensor values and uncalibrated depth all suffice; layers that restructure actions or pre-digest observations mainly add cost.

E.g., relative targets / semantic skills / affordances: none won

Useful info: targeted + compact

Targets the model's frequent errors (empty grasps, mislocalized objects) and is compact enough to afford at every decision.

E.g., full JSON readings triggered context compaction in 38% of first-round episodes

Takeaway 1

Lean harness goes far

29.7 → 38.0

Takeaway 2

Reuse is no free lunch

No clear gain, higher cost

Takeaway 3

Scene geometry helps most

Empty grasps 41 → 16

Takeaway 4

Long-tail failures remain

Mostly task-specific

Discussion

Limits & verdict: a solid “subtractive” study; small gaps need care

Limitations stated by the authors

- Simulation only, one benchmark, one model; each config run once on the eval set
- Path-wise, not full-factorial, ablation: only tests usefulness on top of kept designs
- Experience experiments use the evaluated tasks; self-improvement ran only 5 rounds, 1 practice per task
- Eval set drawn from the authors' own default-testbed history; Generalization uses 75% random layouts (official: 50%); not leaderboard scores

Reviewer's Observations

- Eval-set std. ≈ 2.6: configs at 39.8 / 39.9 / 42.1 / 42.4 are indistinguishable; only 29.7 → 38.0 and 42.4 → 50.3 are reliable
- The last step bundles “code coordinates” with “no decision cap”, yet the action-sequence study shows the budget itself matters a lot
- “Simple” ≠ cheap: per-episode cost lower bound rises from ~$3.8 to $12.5
- Its value: correcting misattributed gains and offering a strong, simple baseline